

作业 6

——d 维 1-median 查询——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。
分享这个文档仅限于学术交流，不用于任何形式的商业用途。

1. 题目

描述

已知3维空间中 n 个点的坐标， q 次询问某一个点到这 n 个点的距离和。对于每次询问，设正确答案为 ANS ，要求输出的解在 $[(1 - e)ANS, (1 + e)ANS]$ 之内，其中 $e = 0.2$ 。

$n \leq 105, q \leq 105$, 坐标范围 $[-10^9, 10^9]$ 。

输入

第一行两个正整数，分别为 n, q 。之后 n 行，每行3个整数。每一行的3个整数描述一个点的坐标。之后 q 行，每行3个整数，每一行的3个整数描述一个需要查询的点的坐标。可能有重点。

输出

q 行，每行一个实数，表示查询的点到其他所有点距离和的估计。

样例输入

```
1 1
1 1 1
1 1 2
```

样例输出

```
1
```

提示

注意 long long 的范围

2. 分析

我们先做一些准备工作：

```
struct point{
    int x, y, z;
    point(){
        cin >> x >> y >> z;
    }
};

double dist(const point& a, const point& b){
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    double dz = a.z - b.z;
    return sqrt(dx * dx + dy * dy + dz * dz);
}
```

这是对作业五的进一步分析讨论。在高中的《概率统计》一章中，我们学过分层随机抽样的概念，很显然，对于方差很大的一组数据，分层随机抽样是更合理的，更容易囊括奇特的数据并且控制它们对抽样的影响程度。对

于本题，我们可以将空间中的点按照距离中心点的远近分成若干层（比如10层），每层随机抽取一定数量的点来估计距离和。这样可以更好地控制误差，尤其是当存在一些点离“中心”很远的情况。

基于分层随机抽样的想法，我们按照以下步骤来实现：

首先，我们需要知道按照什么标准来分层。一个自然的选择是按照点到某个中心点（比如所有点的平均位置）的距离来分层。我们可以计算每个点到这个中心点的距离，然后根据距离将点分成若干层。于是，我们第一步就是要找到这个所谓的圆心。同样，我们不需要非常精确的圆心，所以策略是：任意选取一些点，然后在这些点里面找到距离其他所有点距离之和最小的点就可以。我们把这样的点视作一个可以接受的“圆心”。

```
int n, q;
cin >> n >> q;
vector<point> originalPoints(n), queryPoints(q);
// 选取中心点（距离和最小的点）
int K = min(n, 100);
srand(time(0));
vector<int> centerID(K);
for(int i = 0; i < K; ++i){
    centerID[i] = rand() % n;
}
double minDis = 1e18;
int FinalCenterID = 0;
for(int i = 0; i < K; ++i){
    double sum = 0;
    for(int j = 0; j < n; ++j){
        sum += dist(originalPoints[centerID[i]], originalPoints[j]);
    }
    if(sum < minDis){
        minDis = sum;
        FinalCenterID = centerID[i];
    }
}
```

接下来，我们计算每个点到这个中心点的距离，并按照距离将点分成若干层。我们可以根据距离的分布情况来动态地确定层数和每层的范围。比如，我们可以先计算所有点到中心点的距离，然后根据这些距离的分布情况来划分层次。这样可以更好地适应数据的实际分布，避免某些层过于稀疏或过于密集。

```
// 1. 计算每个点到中心点的距离
vector<double> dis(n);
double cen_sum = 0.0;
for(int i = 0; i < n; i++){
    dis[i] = dist(originalPoints[FinalCenterID], originalPoints[i]);
    cen_sum += dis[i];
}
// 2. 生成半径序列（从平均距离的0.1倍开始，每次翻倍，直到覆盖最远点）
const double eps = 0.1;
vector<double> r;
r.push_back(eps * cen_sum / n);
while (r.back() * 2 < *max_element(dis.begin(), dis.end())) {
    r.push_back(r.back() * 2);
}
// 确保最后一个半径能包含所有点
if (r.back() < *max_element(dis.begin(), dis.end())) {
    r.push_back(r.back() * 2);
}
// 3. 按距离排序索引
vector<int> idx(n);
for(int i = 0; i < n; i++) idx[i] = i;
```

```

sort(idx.begin(), idx.end(), [&](int i, int j) { return dis[i] < dis[j]; });

// 4. 动态分层（每一层对应一个半径区间）
vector<vector<int>> Layer;
int ptr = 0;
for (int i = 0; i < (int)r.size() && ptr < n; i++) {
    vector<int> tmp;
    while (ptr < n && dis[idx[ptr]] <= r[i] + 1e-9) {
        tmp.push_back(idx[ptr]);
        ptr++;
    }
    if (!tmp.empty()) Layer.push_back(tmp);
}
// 如果有点未被覆盖（理论上不会），加入最后一层
if (ptr < n) {
    vector<int> tmp;
    while (ptr < n) tmp.push_back(idx[ptr++]);
    Layer.push_back(tmp);
}

```

最后，我们按照距离将点分成若干层，并在每层中随机抽取一定数量的点来估计距离和。对于每个查询点，我们可以根据它到中心点的距离来确定它属于哪个层，然后在该层中进行随机抽样来估计距离和。

```

// 处理查询（每层最多采样300个点，按层大小加权）
for(int i = 0; i < q; i++){
    double ans = 0;
    for(auto& layer : Layer){
        int sz = layer.size();
        if(sz == 0) continue;
        double layer_sum = 0;
        int sample_cnt = min(300, sz);
        for(int j = 0; j < sample_cnt; j++){
            int idx = layer[rand() % sz];
            layer_sum += dist(queryPoints[i], originalPoints[idx]);
        }
        double layer_avg = layer_sum / sample_cnt;
        ans += layer_avg * sz;    // 加权：层平均距离 × 层大小
    }
    cout << ans << '\n';
}

```

3. 最后附上完整的代码

```

#include <bits/stdc++.h>
using namespace std;
using LL = long long;
struct point{
    int x, y, z;
    point(){
        cin >> x >> y >> z;
    }
};
double dist(const point& a, const point& b){
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    double dz = a.z - b.z;
    return sqrt(dx * dx + dy * dy + dz * dz);
}
int main(){

```

```

int n, q;
cin >> n >> q;
vector<point> originalPoints(n), queryPoints(q);
// 选取中心点（距离和最小的点）
int K = min(n, 100);
srand(time(0));
vector<int> centerID(K);
for(int i = 0; i < K; ++i){
    centerID[i] = rand() % n;
}
double minDis = 1e18;
int FinalCenterID = 0;
for(int i = 0; i < K; ++i){
    double sum = 0;
    for(int j = 0; j < n; ++j){
        sum += dist(originalPoints[centerID[i]], originalPoints[j]);
    }
    if(sum < minDis){
        minDis = sum;
        FinalCenterID = centerID[i];
    }
}

// 1. 计算每个点到中心点的距离
vector<double> dis(n);
double cen_sum = 0.0;
for(int i = 0; i < n; i++){
    dis[i] = dist(originalPoints[FinalCenterID], originalPoints[i]);
    cen_sum += dis[i];
}

// 2. 生成半径序列（从平均距离的0.1倍开始，每次翻倍，直到覆盖最远点）
const double eps = 0.1;
vector<double> r;
r.push_back(eps * cen_sum / n);
while (r.back() * 2 < *max_element(dis.begin(), dis.end())) {
    r.push_back(r.back() * 2);
}
// 确保最后一个半径能包含所有点
if (r.back() < *max_element(dis.begin(), dis.end())) {
    r.push_back(r.back() * 2);
}

// 3. 按距离排序索引
vector<int> idx(n);
for(int i = 0; i < n; i++) idx[i] = i;
sort(idx.begin(), idx.end(), [&](int i, int j) { return dis[i] < dis[j]; });

// 4. 动态分层（每一层对应一个半径区间）
vector<vector<int>> Layer;
int ptr = 0;
for (int i = 0; i < (int)r.size() && ptr < n; i++) {
    vector<int> tmp;
    while (ptr < n && dis[idx[ptr]] <= r[i] + 1e-9) {
        tmp.push_back(idx[ptr]);
        ptr++;
    }
    if (!tmp.empty()) Layer.push_back(tmp);
}

```

```

    }
    // 如果有点未被覆盖（理论上不会），加入最后一层
    if (ptr < n) {
        vector<int> tmp;
        while (ptr < n) tmp.push_back(idx[ptr++]);
        Layer.push_back(tmp);
    }

    // 处理查询（每层最多采样300个点，按层大小加权）
    for(int i = 0; i < q; i++){
        double ans = 0;
        for(auto& layer : Layer){
            int sz = layer.size();
            if(sz == 0) continue;
            double layer_sum = 0;
            int sample_cnt = min(300, sz);
            for(int j = 0; j < sample_cnt; j++){
                int idx = layer[rand() % sz];
                layer_sum += dist(queryPoints[i], originalPoints[idx]);
            }
            double layer_avg = layer_sum / sample_cnt;
            ans += layer_avg * sz;    // 加权：层平均距离 × 层大小
        }
        cout << ans << '\n';
    }

    return 0;
}

```